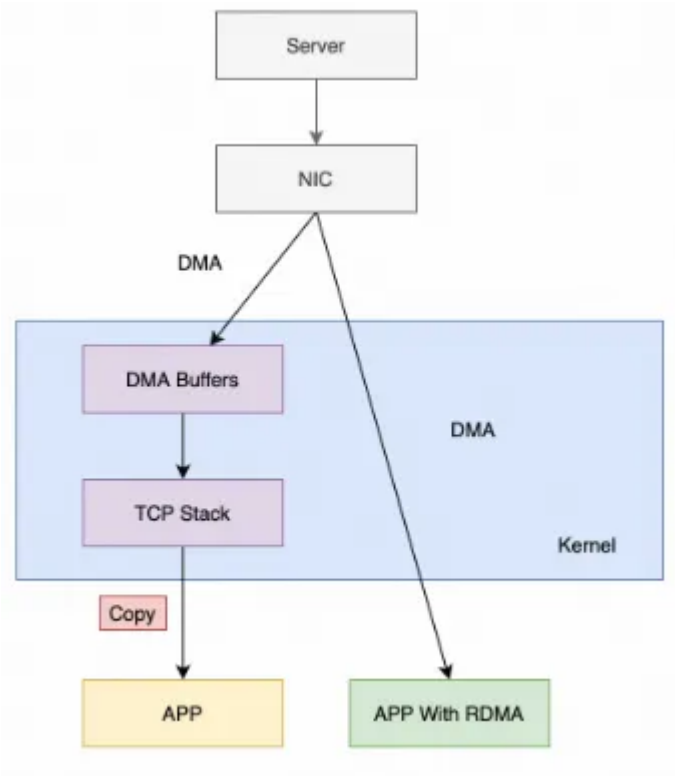
RDMA学习

1. 前言

随着数据中心的飞速发展，高性能网络不断挑战着带宽与时延的极限，网卡带宽从过去的 10 Gb/s 、25 Gb/s 到如今的 100 Gb/s、200 Gb/s 再到下一代的 400Gb/s 网卡，其发展速度已经远大于 CPU 发展的速度。为了满足高性能网络下的通信需求，阿里云不仅自研了高性能用户态协议栈 （Luna、Solar） ，也大规模使用了 RDMA 技术，以充分利用高性能网络。尤其是在存储和 AI 领域，RDMA 被广泛使用。相比于 Kernel TCP 提供的 Socket 接口，RDMA 的抽象更为复杂，为了更好的使用 RDMA，了解其工作原理和机制是必不可少的。本文以 NVIDIA （原 Mellanox）的 RDMA 网卡为例，分析其工作原理和软硬件交互的机制。

2. RDMA 是什么

RDMA (Remote Direct Memory Access) 技术全称远程直接内存访问，是为了解决网络传输中服务器端数据处理的延迟而产生的。RDMA通过网络将数据从一个系统快速移动到另一个系统中，而不需要消耗计算机的处理能力。它消除了内存拷贝和上下文切换的开销，因而能解放内存带宽和 CPU 周期用于提升系统的整体性能。



先看看最常见的 Kernel TCP，其收数据的流程主要要经过以下阶段：

1. 网卡驱动从内核分配 dma buffer，填入收队列
2. 网卡收到数据包，发起 DMA，写入收队列中的 dma buffer
3. 网卡产生中断
4. 网卡驱动查看收队列，取出 dma buffer，交给协议栈
5. 协议栈处理报文
6. 操作系统通知用户态程序有可读事件

7. 用户态程序准备 buffer，发起系统调用
8. 内核拷贝数据至用户态程序的 buffer 中
9. 系统调用结束

可以发现，上述流程有**三次上下文切换**（中断上下文切换、用户态与内核态上下文切换），有**一次内存拷贝**。虽然内核有一些优化手段，比如通过 NAPI 机制减少中断数量，但是在高性能场景下， Kernel TCP 的延迟和吞吐的表现依然不佳。

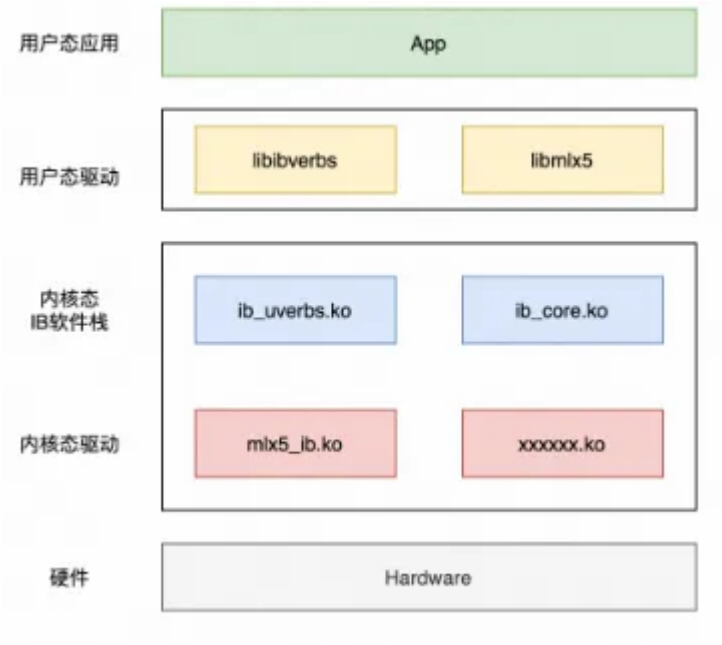
使用 RDMA 技术后，收数据的主要流程变为（以send/recv为例）：

1. 用户态程序分配 buffer，填入收队列
2. 网卡收到数据包，发起 DMA，写入收队列中的 buffer
3. 网卡产生完成事件（可以不产生中断）
4. 用户态程序 polling 完成事件
5. 用户态程序处理 buffer

上述流程**没有上下文切换，没有数据拷贝，没有协议栈的处理逻辑**（卸载到了RDMA网卡内），也没有内核的参与。CPU 可以专注处理数据和业务逻辑，不用花大量的 cycles 去处理协议栈和内存拷贝。

3. RDMA 的软件架构

从上述分析可以看出，RDMA 和传统的内核协议栈是完全独立的，因此其软件架构也与内核协议栈很不一样，包含以下部分：



1. 用户态驱动（libibverbs、libmlx5等）：这些库都属于 rdma-core 项目。为用户提供各种 Verbs API，另外也有一些厂商的特有 API。之所以这层被称为“用户态驱动”，是因为 RDMA 要在用户态直接和硬件打交道，传统在内核态实现 HAL 的方式不满足 RDMA 的需求，因此需要在用户态感知硬件细节。
2. 内核态IB软件栈：内核态的一层抽象，对应用提供统一的接口。这些接口不仅用户态可以调用，内核态也可以调用。
3. 内核态驱动：各个厂商实现的网卡驱动，直接和硬件交互。

4. RDMA 的内存管理

由于 RDMA 要直接让硬件读写用户态程序的内存，这带来了很多问题：

1. 安全问题：用户态程序能否利用网卡读写**任意物理内存**？
2. 地址映射问题：用户态程序使用的是虚拟地址，实际的物理地址是操作系统管理的。网卡怎么知道**虚拟地址和物理地址**的映射关系？
3. 地址映射会变化：操作系统可能会对内存做 swap、压缩，操作系统还有一些复杂的机制，比如 Copy-On-Write。这些情况下，怎么保证网卡访问的**地址的正确性**？
4.

为了解决上述问题，RDMA 引入了两个概念：

1. PD（Protection Domain）：在 RDMA 中，PD 是一个容纳了各种资源的“容器”，类似一个租户 ID，将这些资源纳入自己的保护范围内，避免他们被未经授权的访问。一个进程中可以创建多个 PD，各个 PD 所容纳的资源彼此隔离，无法一起使用。
2. MR（Memory Registration）：RDMA 中对内存保护的一种措施，**只有将要操作的内存注册到 MR 中，这段内存才能被 RDMA 使用。**MR包括了 PD、lkey、rkey、地址、长度、权限这几个属性。PD 是这个 MR 所属的保护域，其他保护域的上下文是不能访问这个 MR 的（避免了暴力遍历内存访问其他进程内存的可能）。lkey、rkey 统称为 mkey 是一个访问内存的凭据（分别对应本地访问和远程访问），RDMA 的所有操作都要有 mkey 才能进行。

这样讲可能会有一些抽象，以一个实际的例子来看看 RDMA 是怎么注册内存的：

```
// 扫描 RDMA 设备，找到我们要用的设备
int num_devices = 0, i;
struct ibv_device ** device_list = ibv_get_device_list(&num_devices);
struct ibv_device *device = NULL;
for (i = 0; i < num_devices; ++i) {
    if (strcmp("mlx5_bond_0", ibv_get_device_name(device_list[i])) == 0) {
        device = device_list[i];
        break;
    }
}

// 打开设备，产生一个 context
struct ibv_context *ibv_ctx = ibv_open_device(device);

// 分配一个 PD
struct ibv_pd* pd = ibv_alloc_pd(ibv_ctx);

// 分配 8k 内存，按照 page 对齐（4k）
void *alloc_region = NULL;
posix_memalign(&alloc_region, sysconf(_SC_PAGESIZE), 8192);

// 注册内存
struct ibv_mr* reg_mr = ibv_reg_mr(pd, alloc_region, mr_len,
IBV_ACCESS_LOCAL_WRITE);
```

打开内核模块的 debug 信息：

```
echo "file mr.c +p" > /sys/kernel/debug/dynamic_debug/control
echo "file mem.c +p" > /sys/kernel/debug/dynamic_debug/control
```

输出的日志：

```
infiniband mlx5_0: mlx5_ib_reg_user_mr:1300:(pid 100804): start 0xdb1000,
virt_addr 0xdb1000, length 0x2000, access_flags 0x100007
infiniband mlx5_0: mr_umem_get:834:(pid 100804): npages 2, ncont 2, order 1,
page_shift 12
infiniband mlx5_0: get_cache_mr:484:(pid 100804): order 2, cache index 0
infiniband mlx5_0: mlx5_ib_reg_user_mr:1369:(pid 100804): mkey 0xdd5d
infiniband mlx5_0: __mlx5_ib_populate_pas:158:(pid 100804): pas[0] 0x3e07a92003
infiniband mlx5_0: __mlx5_ib_populate_pas:158:(pid 100804): pas[1] 0x3e106a6003
```

结合内核模块的代码分析，内存注册大致有这几个阶段：

1. 用户态程序调用 ibv_reg_mr，通过 uverbs 接口发到内核（实际用的系统调用为 ioctl或者 write），操作的文件是 /dev/infiniband/uverbsX
2. 经过 uverbs 层的转换后，最终调用到设备驱动的代码，NVIDIA 网卡上是 mlx5_ib_reg_user_mr 这个函数：

```
err = mr_umem_get(dev, start, length, access_flags, &umem,
            &npages, &page_shift, &ncont, &order, true);

if (err < 0)
    return ERR_PTR(err);

use_umr = mlx5_ib_can_use_umr(dev, IB_ACCESS_DISABLE_RELAXED_ORDERING,
                  access_flags);

if (umem->is_peer && MLX5_CAP_GEN(dev->mdev, ats)) {
    use_umr = false;
} else if (order <= mr_cache_max_order(dev) && use_umr) {
    mr = alloc_mr_from_cache(pd, umem, virt_addr, length, ncont,
                 page_shift, order, access_flags);
    if (PTR_ERR(mr) == -EAGAIN) {
        mlx5_ib_dbg(dev, "cache empty for order %d\n", order);
        mr = NULL;
    }
} else if (!MLX5_CAP_GEN(dev->mdev, umr_extended_translation_offset)) {
    if (access_flags & IB_ACCESS_ON_DEMAND) {
        err = -EINVAL;
        pr_err("Got MR registration for ODP MR > 512MB, not supported for Connect-IB\n");
        goto error;
    }
    use_umr = false;
}

if (!mr) {
    mutex_lock(&dev->slow_path_mutex);
    mr = reg_create(NULL, pd, virt_addr, length, umem, ncont,
            page_shift, access_flags, !use_umr);
    mutex_unlock(&dev->slow_path_mutex);
}
```

这个函数主要做了两件事情，通过 mr_umem_get 将用户态内存 pin 住，并拿到物理地址。以及通过 reg_create 向网卡发送创建 mkey 的命令。mr_umem_get 最终会调用到 ib_core.ko 中的 __ib_umem_get 函数：

```
/**
 * __ib_umem_get - Pin and DMA map userspace memory.
 *
 * @device: IB device to connect UMEM
 * @addr: userspace virtual address to start at
 * @size: length of region to pin
 * @access: IB_ACCESS_xxx flags for memory being pinned
 * @peer_mem_flags: IB_PEER_MEM_xxx flags for memory being used
 */
struct ib_umem *__ib_umem_get(struct ib_device *device,
                              unsigned long addr, size_t size, int access,
                              unsigned long peer_mem_flags)
{
```

这个函数会通过内核的 pin_user_pages_fast 接口，防止用户内存的映射关系发生意外的改变（比如 swap）。

reg_create 会按照网卡开发手册定义的命令格式，发送创建 mkey 的命令：

```
u32 lout[MLX5_ST_SZ_DW(create_mkey_out)] = {};
u32 mkey_index;
void *mkc;
int err;

MLX5_SET(create_mkey_in, in, opcode, MLX5_CMD_OP_CREATE_MKEY);

err = mlx5_cmd_exec(dev, in, inlen, lout, sizeof(lout));
if (err)
    return err;

mkc = MLX5_ADDR_OF(create_mkey_in, in, memory_key_mkey_entry);
mkey_index = MLX5_GET(create_mkey_out, lout, mkey_index);
mkey->iova = MLX5_GET64(mkc, mkc, start_addr);
mkey->size = MLX5_GET64(mkc, mkc, len);
mkey->key = (u32)mlx5_mkey_variant(mkey->key) | mlx5_idx_to_mkey(mkey_index);
mkey->pd = MLX5_GET(mkc, mkc, pd);

mlx5_core_dbg(dev, "out 0x%x, mkey 0x%x\n", mkey_index, mkey->key);
```

经过上述流程，最终会在网卡中创建一个 mkey 对象：

```
mkey 0xdd5d
virt_addr 0xdb1000, length 0x2000, access_flags 0x100007
pas[0] 0x3e07a92003
pas[1] 0x3e106a6003
```

网卡收到虚拟地址的时候，就能够知道实际的物理页是什么，并且可以做权限控制。

现在可以回答上面的问题了：

1. 安全问题：用户态程序能否利用网卡读写任意物理内存？

不能，RDMA 通过 PD 和 MR 机制做了严格的内存保护。

2. 地址映射问题：用户态程序使用的是虚拟地址，实际的物理地址是操作系统管理的。网卡怎么知道虚拟地址和物理的映射关系？

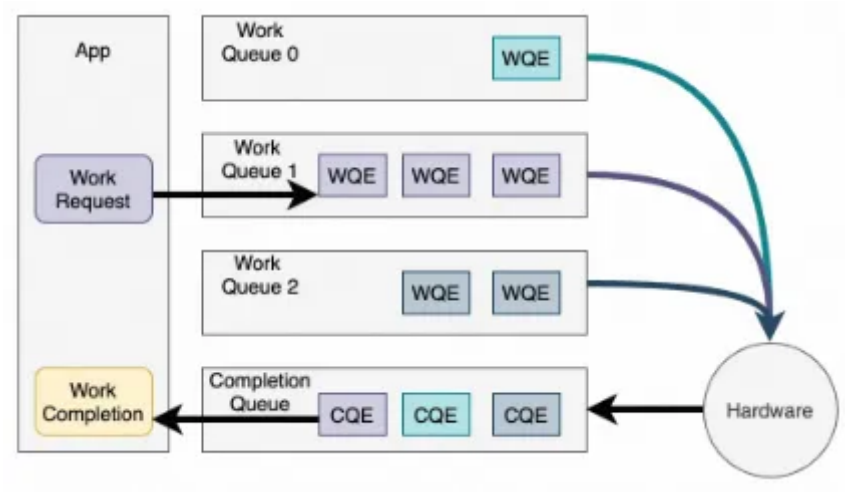
驱动会告诉网卡映射关系，后续数据流中，网卡自己转换。

3. 地址映射会变化：操作系统可能会对内存做 swap、压缩，操作系统还有一些复杂的机制，比如 Copy-On-Write。这些情况下，怎么保证网卡访问的地址的正确性？

通过驱动调用【pin_user_pages_fast】保障。另外，用户态驱动会给注册的内存打上 DONT_FORK 的标志，避免 Copy-On-Write 发生。

5. RDMA 的软硬交互

RDMA 的软硬交换的基础单元是 Work Queue。Work Queue 是一个单生产者单消费者的环形队列。Work Queue 根据功能不同，主要分为 SQ（发送）、RQ（接收）、CQ（完成）和 EQ（事件）等。



发送一个 RDMA 请求的大致流程为：

1. 软件构造 WQE （Work Queue Element），提交至 Work Queue 中
2. 软件写 Doorbell 通知硬件
3. 硬件拉取 WQE，处理 WQE
4. 硬件处理完成，产生 CQE，写入 CQ
5. 硬件产生中断（可选）
6. 软件 **Polling** CQ
7. 软件读取硬件更新后的 CQE，得知 WQE 完成

还是以一个实际的例子进行分析，这样才能深入理解全流程：

```
uint32_t send_demo(struct ibv_qp *qp, struct ibv_mr *mr)
{
    struct ibv_send_wr sq_wr = {}, *bad_wr_send = NULL;
    struct ibv_sge sq_wr_sge[1];

    // 把 mr 的数据发送出去
    sq_wr_sge[0].lkey = mr->lkey;
    sq_wr_sge[0].addr = (uint64_t)mr->addr;
    sq_wr_sge[0].length = mr->length;

    sq_wr.next = NULL;
    sq_wr.wr_id = 0x31415926; // 请求的ID，收到 WC 的时候可以对应上
    sq_wr.send_flags = IBV_SEND_SIGNALED; // 告诉网卡这个请求给我产生 CQE
```

```
    sq_wr.opcode = IBV_WR_SEND;
    sq_wr.sg_list = sq_wr_sge;
    sq_wr.num_sge = 1;

    ibv_post_send(qp, &sq_wr, &bad_wr_send); // 提交请求到网卡

    struct ibv_wc wc = {};
    while (ibv_poll_cq(qp->send_cq, 1, &wc) == 0) {
        continue;
    }

    if (wc.status != IBV_WC_SUCCESS || wc.wr_id != 0x31415926) {
        exit(__LINE__);
    }

    return 0;
}
```

这段代码的核心是 ibv_post_send 和 ibv_poll_cq，ibv_post_send 的用途是向网卡提交请求，**ibv_poll_cq 的用途是轮询 CQ**，看看是否有请求完成。

另外，IB 规范定义了很多抽象：

1. ibv_send_wr 这个是 SQ 的 WQE 的抽象（不同厂商的网卡 WQE 格式不同）
2. ibv_sge 是数据的抽象，包含 （addr，len，lkey） 三个字段
3. ibv_wc 是 CQE 的抽象，通过 wr_id 和 ibv_send_wr 对应起来

把 rdma-core 的 debug 打开，可以看到这样的两条日志：

```
dump wqe at 0x7fc59b357000
0000000a 00115003 00000008 00000000
00000000 c0000000 00000000 00000000
0000022a 00008a0b 00007fc5 9a5b2000

mlx5_get_next_cqe:565: dump cqe for cqn 0x4c2:
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00004863 2a39b102 0a001150 00003f00
```

第一条就是我们调用 ibv_post_send 后，rdma-core 事件向网卡提交的 WQE。

第二条是我们调用 ibv_poll_cq 后，收到网卡给我们的 CQE。

在 NVIDIA 网卡上，ibv_post_send 最终调用的是 _mlx5_post_send：

```
        mlx5_opcode = mlx5_ib_opcode[wr->opcode];
        ctrl->opmod_idx_opcode = htobe32(((qp->sq.cur_post & 0xffff) << 8) |
                                         mlx5_opcode                      |
                                         (opmod << 24));
        ctrl->qpn_ds = htobe32(size | (ibqp->qp_num << 8));

        if (unlikely(qp->wq_sig))
            ctrl->signature = wq_sig(ctrl);

        qp->sq.wrid[idx] = wr->wr_id;
        qp->sq.wqe_head[idx] = qp->sq.head + nreq;
        qp->sq.cur_post += DIV_ROUND_UP(size * 16, MLX5_SEND_WQE_BB);

#ifdef MLX5_DEBUG
        if (mlx5_debug_mask & MLX5_DBG_QP_SEND)
            dump_wqe(to_mctx(ibqp->context), idx, size, qp);
#endif
    }

out:
    qp->fm_cache = next_fence;
    post_send_db(qp, bf, nreq, inl, size, ctrl);

    mlx5_spin_unlock(&qp->sq.lock);
```

这个函数的核心功能是：

1. 把 ibv_send_wr 翻译为 NVIDIA 网卡的 WQE 格式
2. Doorbell 通知网卡

5.1 填充 WQE

根据 NVIDIA 的手册，一个 Send 请求的 WQE 包含两个部分：

控制段：

Table 56 - General - Ctrl Segment Format

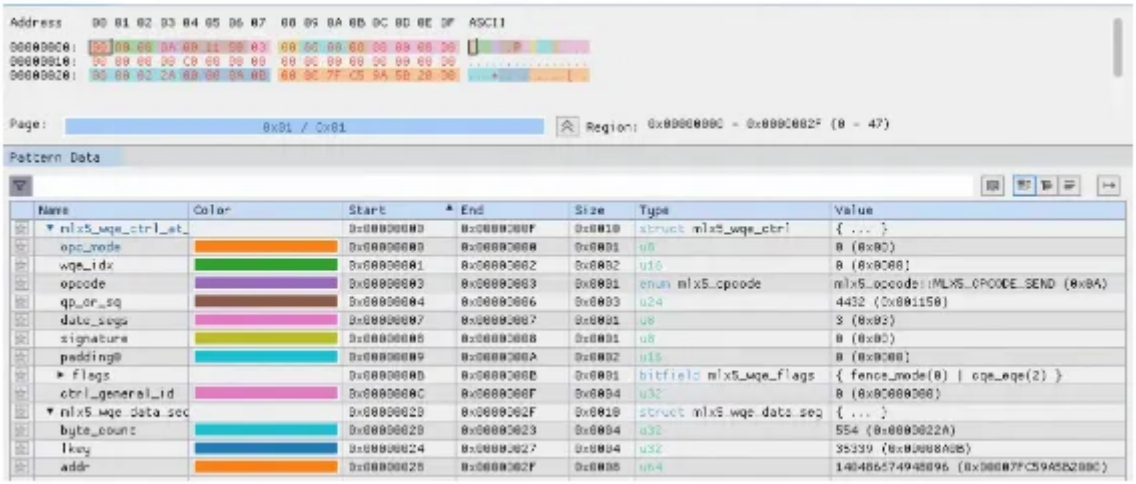
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
OPC_MOD								wqe_index																OPCODE								00h
qp_or_sq																									DS							04h
signature																								FM			CE			SE		08h
ctrl_general_id																																0Ch

数据段：

Table 89 - Memory Pointer Data Segment Format

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
is_inline	byte_count																															00
l_key																																04
local_address																																08-0Ch

根据手册的定义，我们解析一下上面日志中的 WQE：



可以看到，和我们软件填的 ibv_send_wr 是相似的。注意到一个细节，我们填的 wr_id 没有出现在 WQE 中，实际上这个 wr_id 是存在 rdma-core 里的。因为 WQE 是按序处理的，收到 CQE 的时候可以找到当初提交的 ibv_send_wr。

5.2 Doorbell 通知网卡

post_send_db 这个函数很简单，但是细节非常多：

```
 * Make sure that descriptors are written before
 * updating doorbell record and ringing the doorbell
 */
udma_to_device_barrier();
qp->db[MLX5_SND_DBR] = htobe32(qp->sq.cur_post & 0xffff);

/* Make sure that the doorbell write happens before the memcpy
 * to WC memory below
 */
ctx = to_mctx(qp->ibv_qp->context);
if (bf->need_lock)
    mmio_wc_spinlock(&bf->lock.lock);
else
    mmio_wc_start();

if (!ctx->shut_up_bf && nreq == 1 && bf->uuarn &&
    (inl || ctx->prefer_bf) && size > 1 &&
    size <= bf->buf_size / 16)
    mlx5_bf_copy(bf->reg + bf->offset, ctrl,
                 align(size * 16, 64), qp);
else
    mmio_write64_be(bf->reg + bf->offset, *(__be64 *)ctrl);
```

首先，这个函数会加一个内存屏障，保证 Doorbell 到达硬件时，前面构建的 WQE 已经可见。

然后，更新 QP 的 Doorbell 信息（其实就是队列的 tail）

最后，通过 mmio_write64_be 写网卡的寄存器，告诉网卡我准备好了，你可以发了。

mmio_write64_be 使用的是 sse 指令（x86上），WQE 的第一部分（上文的控制段）写入网卡的寄存器：

实际上就是一个赋值操作，等效于：

```
*(uint64_t)(bf->reg + bf->offset) = *(uint64_t *)ctrl;
```

使用 SSE 主要是为了保证原子性，避免网卡先收到前4字节，再收到后4字节。

看来所有的魔法都在 bf->reg 上，这个 buffer 是什么？为什么软件往这个 buffer 写一个 uint64，网卡就能收到事件？

gdb 中可以看到这个 bf->reg 看起来是一个平平无奇的虚拟地址，但是 gdb 读取会失败。

```
(gdb) p *bf
$2 = {reg = 0x7ffff7fea800, need_lock = 0, lock = {lock = 1, in_use = 0, need_lock = 0}, offset = 0, buf_size = 256,
  uuarn = 24, uar_mmap_offset = 24576, uar = 0x0, bfreg_dyn_index = 0, devx_uar = {dv_devx_uar = {reg_addr = 0x0,
      base_addr = 0x0, page_id = 0, mmap_off = 0, comp_mask = 0}, context = 0x0}}
(gdb) x 0x7ffff7fea800
0x7ffff7fea800: Cannot access memory at address 0x7ffff7fea800
(gdb)
```

看看进程的 maps 信息：

```
7ffff7fe8000-7ffff7fe9000 r--s 00500000 00:05 115759                     /dev/infiniband/uverbs0
7ffff7fe9000-7ffff7fea000 -w-s 00007000 00:05 115759                     /dev/infiniband/uverbs0
7ffff7fea000-7ffff7feb000 -w-s 00006000 00:05 115759                     /dev/infiniband/uverbs0
7ffff7feb000-7ffff7fec000 -w-s 00005000 00:05 115759                     /dev/infiniband/uverbs0
7ffff7fec000-7ffff7fed000 -w-s 00004000 00:05 115759                     /dev/infiniband/uverbs0
7ffff7fed000-7ffff7fee000 -w-s 00003000 00:05 115759                     /dev/infiniband/uverbs0
7ffff7fee000-7ffff7fef000 -w-s 00002000 00:05 115759                     /dev/infiniband/uverbs0
7ffff7fef000-7ffff7ff0000 -w-s 00001000 00:05 115759                     /dev/infiniband/uverbs0
7ffff7ff0000-7ffff7ff1000 -w-s 00000000 00:05 115759                     /dev/infiniband/uverbs0
```

这段内存被映射成 Write Only 了，所以读取失败。那这段内存是什么呢？

用 crash 看看页表吧：

```
crash> vtop 0x7ffff7fea800
VIRTUAL     PHYSICAL
7ffff7fea800  1bffc020800

   PML: 50fb4627f8 => 2d4d14c067
   PUD: 2d4d14cff8 => 2f03936067
   PMD: 2f03936df8 => 372e6ee067
   PTE: 372e6eef50 => 800001bffc02022f
  PAGE: 1bffc020000

      PTE          PHYSICAL    FLAGS
800001bffc02022f  1bffc020000  (PRESENT|RW|USER|PWT|ACCESSED|NX)

      VMA           START       END     FLAGS FILE
ffff885e5338b290 7ffff7fea000 7ffff7feb000 50444fa /dev/infiniband/uverbs0
```

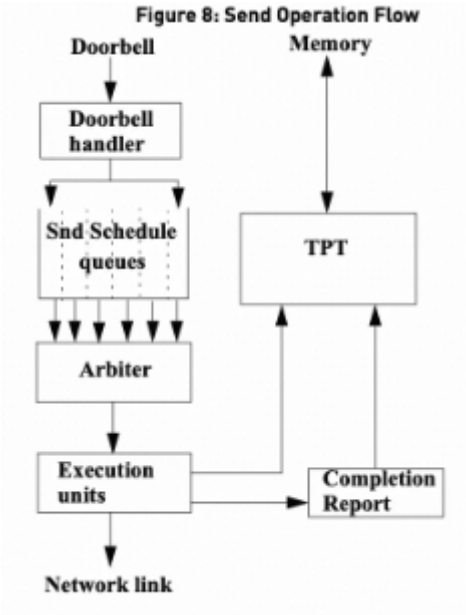
物理地址是 1bffc020800，通过 /proc/iomem 可以看到这个物理地址是属于 0000:3b:00.0 这个 PCI 设备的：

```
18000000000-1bfffffffff : PCI Bus 0000:3a
  18000000000-180001fffff : PCI Bus 0000:3c
  1bffa000000-1bffeffffff : PCI Bus 0000:3b
    1bffa000000-1bffbffffff : 0000:3b:00.1
      1bffa000000-1bffbffffff : mlx5_core
    1bffc000000-1bffdffffff : 0000:3b:00.0
      1bffc000000-1bffdffffff : mlx5_core
    1bffe000000-1bffe7fffff : 0000:3b:00.1
    1bffe800000-1bffeffffff : 0000:3b:00.0
1c000000000-1ffffffffff : PCI Bus 0000:5d
  1c000000000-1c0001fffff : PCI Bus 0000:5e
20000000000-23fffffffff : PCI Bus 0000:80
```

这个设备正是我们用的 RDMA 网卡，这个地址位于 BAR0：

```
$sudo lspci -vvv -s 0000:3b:00.0
3b:00.0 Ethernet controller: Mellanox Technologies MT27710 Family [ConnectX-4 Lx]
        Subsystem: Mellanox Technologies Device 0021
        Physical Slot: 2-1
        Control: I/O- Mem+ BusMaster+ SpecCycle- MemWINV- VGASnoop- ParErr- Stepping- SERR- FastB2B- DisINTx+
        Status: Cap+ 66MHz- UDF- FastB2B- ParErr- DEVSEL=fast >TAbort- <TAbort- <MAbort- >SERR- <PERR- INTx-
        Latency: 0, Cache Line Size: 64 bytes
        Interrupt: pin A routed to IRQ 51
        NUMA node: 0
        Region 0: Memory at 1bffc000000 (64-bit, prefetchable) [size=32M]
        Expansion ROM at b8600000 [disabled] [size=1M]
        Capabilities: [60] Express (v2) Endpoint, MSI 00
                DevCap: MaxPayload 512 bytes, PhantFunc 0, Latency L0s unlimited, L1 unlimited
                        ExtTag+ AttnBtn- AttnInd- PwrInd- RBE+ FLReset+ SlotPowerLimit 0.000W
                DevCtl: Report errors: Correctable- Non-Fatal- Fatal- Unsupported-
                        RlxdOrd+ ExtTag+ PhantFunc- AuxPwr- NoSnoop+ FLReset-
                        MaxPayload 256 bytes, MaxReadReq 256 bytes
```

根据 NVIDIA 的手册，软件写的这个地址区域被称为 UAR（User Access Region）。**UAR 是 PCI 地址空间的一部分，被网卡驱动直接映射至应用的内存中。**当软件往 UAR 写 Doorbell 后，网卡会从 PCI 上收到报文，从而获知软件需要发送数据了这件事情。



随后，网卡会经过调度和仲裁，拉取 WQE 进行执行。在执行过程中，网卡会先从 WQE 中提取 mkey 和地址，然后送进 TPT （Translation and Protection）单元中查询 mkey 的信息，拿到地址和权限，校验通过后进行 DMA，最后把数据发送至物理接口。当数据发送完成，且被对端确认后，网卡会产生 CQE 通知软件。软件收到 CQE 后才可以进行资源的回收。这点和 Socket 接口不同，Socket 接口是有拷贝的，软件调用 write 结束后，就可以把资源回收，而 RDMA 是零拷贝的，资源的生命周期管理会更加复杂。

6. 总结

至此，RDMA 基本的工作原理和软硬件交互的机制就介绍完了。RDMA 技术的出发点很简单，就是能不能饶过一切限制，直接把数据送到用户手中。在此之上，做了非常多的工作，成为了一个非常复杂的技术。RDMA 也不是银弹，有着编程复杂、调试复杂、硬件依赖重、不能长传等问题。对于 IO 密集型的业务，RDMA 可以节约很多 CPU，获得高吞吐低延迟的性能；而对于计算密集的业务，或者追求硬件无关的业务，TCP 是更合适的选择。